

Transformer

Attention Is All You Need

學習主題

1. 什麼是 sequence-to-sequence 任務?
2. Encoder 與 Decoder
3. Self-Attention
4. Positional Encoding
5. Residual 與 Layer Normalization
6. 實作範例：文字接龍
7. 實作範例：文字翻譯

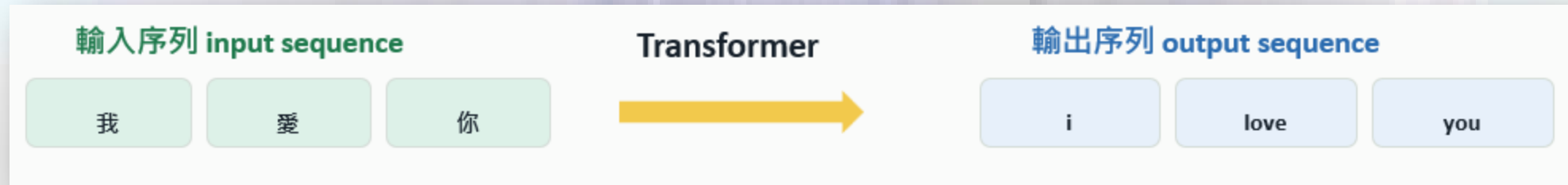
```
class TinyTransformerLM(nn.Module):
    def __init__(self, vocab_size, d_model=64, nhead=4, num_layers=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead,
            dim_feedforward=128, dropout=0.1, batch_first=True
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers
        )
        self.fc = nn.Linear(d_model, vocab_size)
    def forward(self, x):
        x = self.embedding(x)
        x = self.pos_encoder(x)
        x = self.transformer(x)
        logits = self.fc(x)
        return logits
```

參考資料：

- Transformer(上): <https://www.youtube.com/watch?v=n9TlOhRjYoc&t=66s>
- Transformer(下): <https://www.youtube.com/watch?v=N6aRv06iv2g>
- Self-Attention(上): <https://www.youtube.com/watch?v=hYdO9CscNes&t=203s>
- Self-Attention(下): <https://www.youtube.com/watch?v=gmsMY5kc-zw>
- Transformer 原始論文：A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” Advances in Neural Information Processing Systems 30, pp. 6000–6010, 2017.

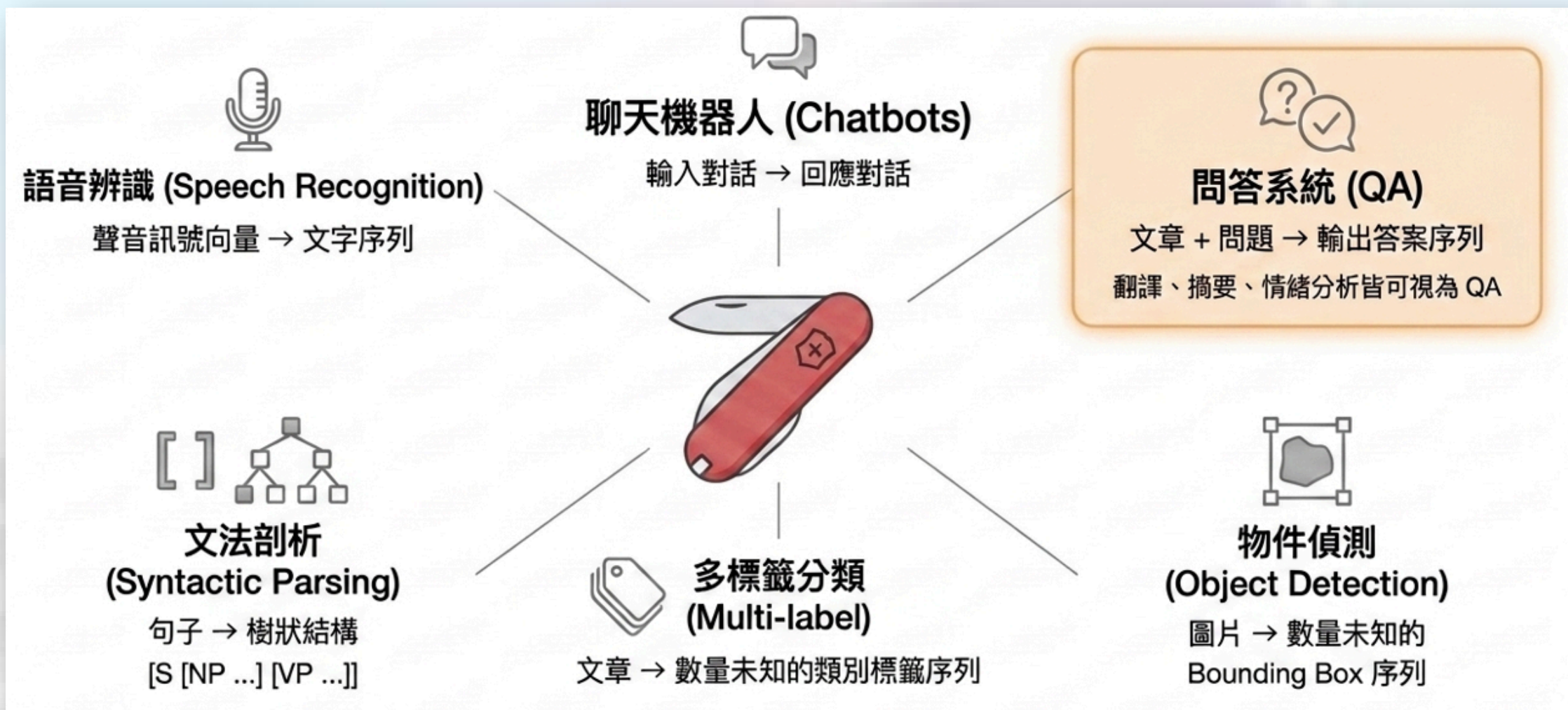
一、Sequence-to-Sequence 任務

- Transformer 主要處理 Sequence-to-Sequence (Seq2Seq) 問題，它可以把一段序列轉成另一段序列。
 - 範例：



- 應用場域：翻譯、語音辨識、語音合成、摘要、問答、聊天、程式碼補全，都可以看成「讀入一段序列，產生另一段序列」。

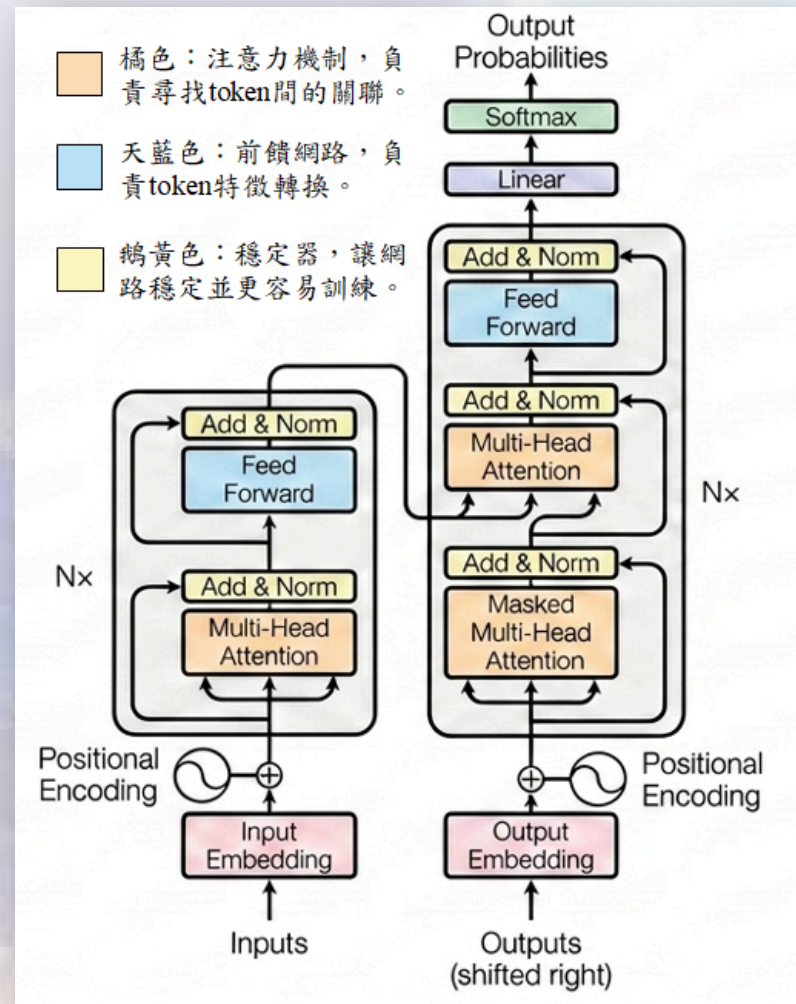
Transformer 可說是深度學習的萬能瑞士刀：萬物皆可 Seq2Seq



二、Encoder 與 Decoder

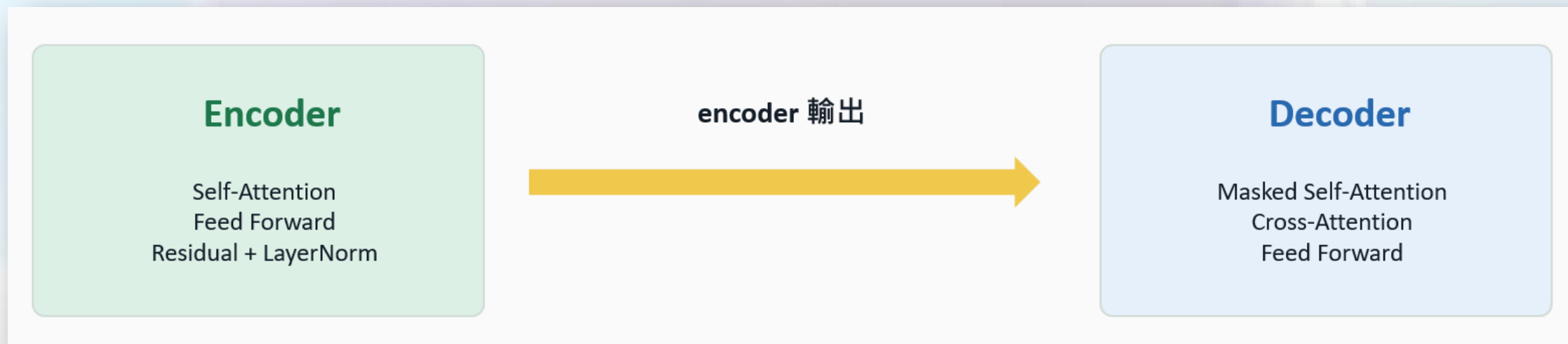
系統架構：

- 左半部：編碼器 (Encoder)
 - 負責理解輸入序列。由N個完全相同的疊加層組成，提煉出具有高度語意代表性的連續向量表示。
- 右半部：解碼器 (Decoder)
 - 負責生成輸出序列。同樣由N個疊加層組成，採用自迴歸(auto-regressive)機制，根據已生成的符號預測下一個符號。



更進一步說，Encoder 讀懂輸入，Decoder 逐步產生輸出：

- Transformer 把理解與生成分成兩個角色，兩者透過 cross-attention 連接



- 直覺說法：
 - Encoder 像閱讀理解：把輸入句子轉成上下文化表示。
 - Decoder 像作文：每次只根據已產生的 token 與 encoder 輸出預測下一個 token。

Encoder block 主要負責建立輸入 token 的上下文表示：

- 每個 token 會在 self-attention 裡和同一句中的其他 token 互相交換資訊。

元件	做什麼	輸入 / 輸出 shape
Token embedding	把 token id 轉成 vector。	[batch, src_len] -> [batch, src_len, d_model]
Positional encoding	加入位置資訊，補足 attention 不知道順序的問題。	shape 不變
Multi-head self-attention	讓每個輸入 token 依內容決定要看其他哪些 token。	shape 不變
Feed-forward network	對每個位置做非線性轉換，提升表示能力。	shape 不變

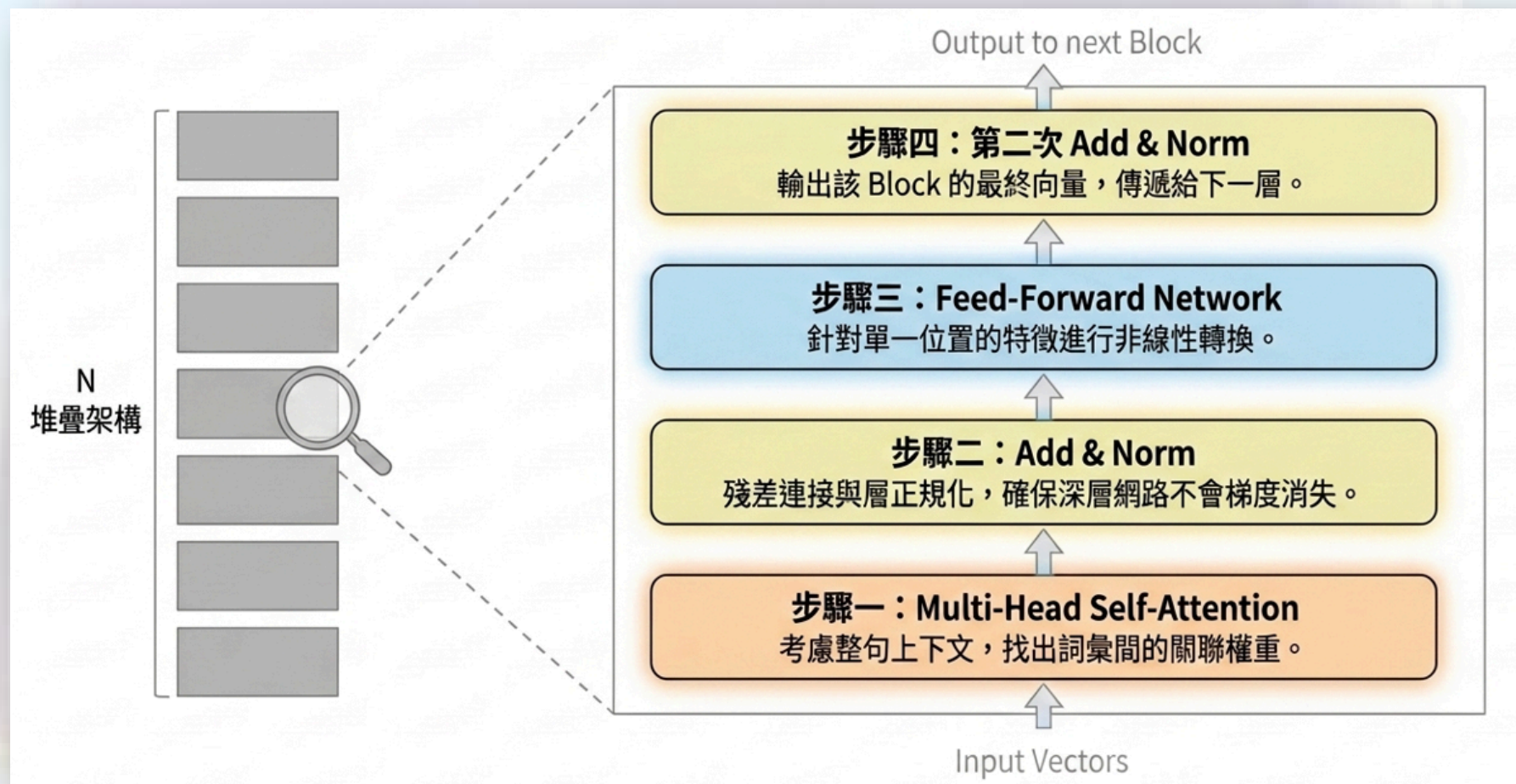
Decoder block 會一邊看已產生的輸出，一邊讀 encoder 輸出：

- 關鍵觀察點：生成第 t 個 token 時，不應該偷看第 $t+1$ 個答案，所以會有 Masked self-attention。

元件	做什麼	為什麼需要
Masked self-attention	只允許 decoder 看見目前與過去 token。	避免訓練時偷看未來答案。
Cross-attention	用 decoder 的 query 去讀 encoder output。	把輸入資訊拿來生成輸出。
Feed-forward network	對每個輸出位置做更深的特徵轉換。	增加非線性與表示能力。
Linear + Softmax	把 hidden vector 轉成下一個 token 的機率。	讓模型可以逐步產生需要的輸出。

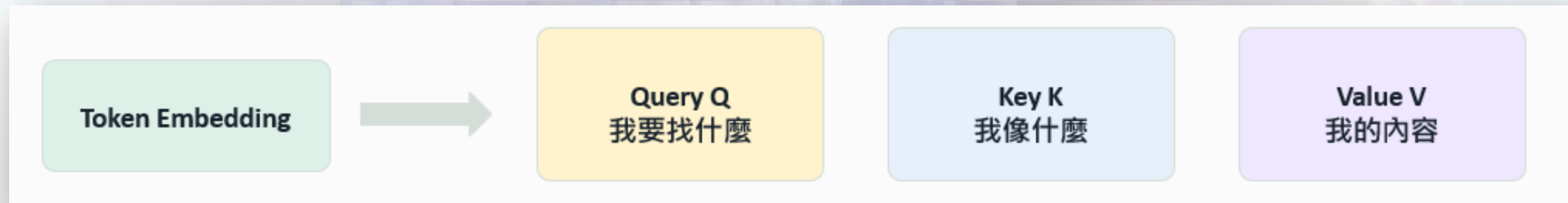
完整 Transformer 會把多個可重複 block 串起來：

- Encoder 與 Decoder block 都會重複 N 次，讓模型逐層建立更抽象的序列表示。



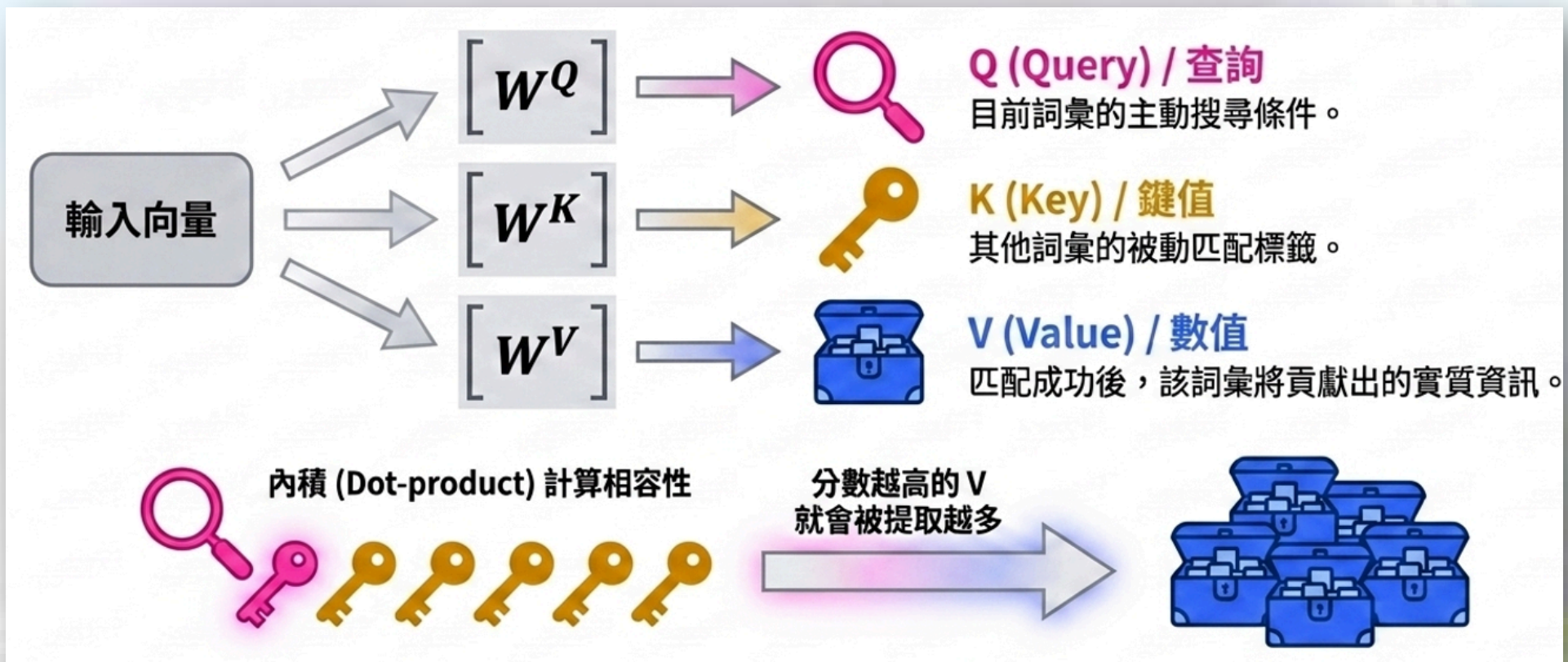
三、Self-Attention

- 每個 token embedding 會被投影成 query (Q)、key (K)、value (V) 三種角色
- 每個 token 會透過這三種角色決定該看誰、看多少、拿到什麼資訊



實際的運作邏輯：

- 每一個輸入向量都會透過學習到的權重矩陣，生成三個專屬角色：Q、K、V。拿自己的 Q 去跟所有人的 K 進行內積，決定提取多少 V 的資訊。



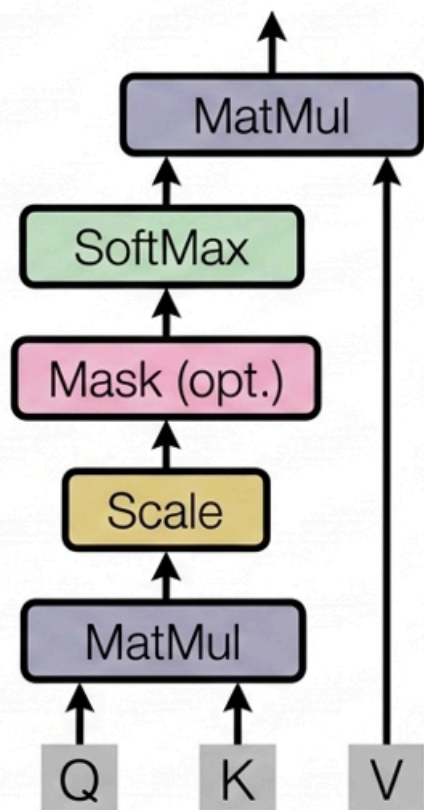
權重矩陣：

- Attention weight 是一個分配：決定目前 token 從其他 token 收集多少資訊
 - 例如「貓」這個 token 可以把一部分注意力放在「喜歡」，因為動詞與受詞之間有語意關係。
 - 模型不是手寫規則，而是從資料中學出這些權重。
 - 這些權重會乘上向量 V ，決定每一個 token 提取多少資訊。

Query token	Key token			
	我	喜	歡	貓
我	0.55	0.15	0.10	0.20
喜	0.10	0.50	0.28	0.12
歡	0.08	0.34	0.45	0.13
貓	0.12	0.18	0.22	0.48

Scaled Dot-Product Attention:

Q, K, V 的配對遊戲： Scaled Dot-Product Attention



Query (Q - 查詢)：
我正在尋找什麼樣的資訊？



Key (K - 鑰匙)：
我身上帶有什麼樣的標籤？

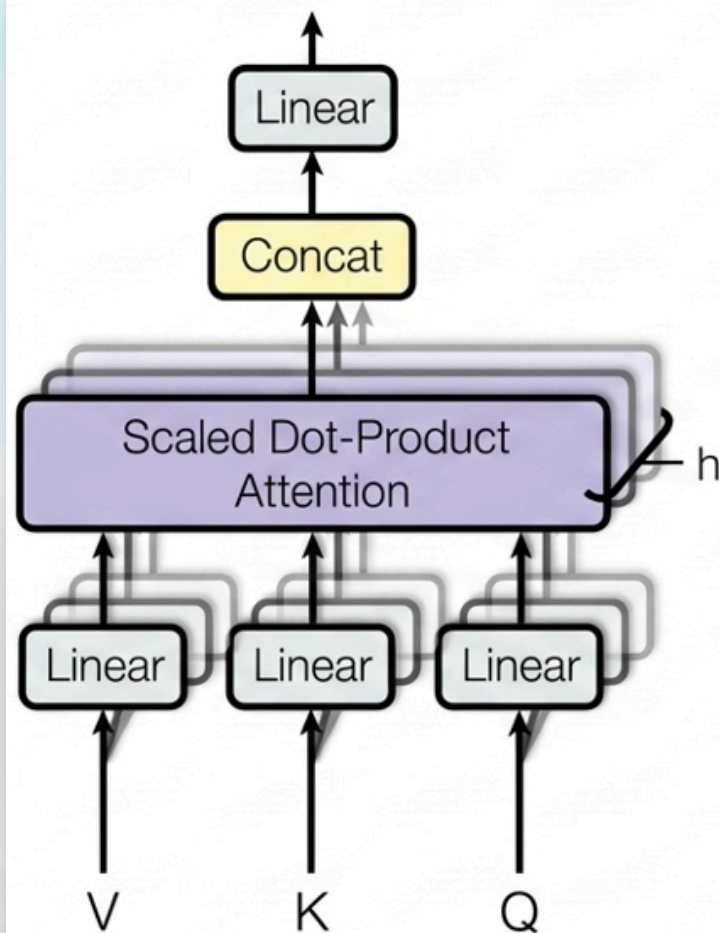


Value (V - 價值)：
我實際包含的資訊內容。

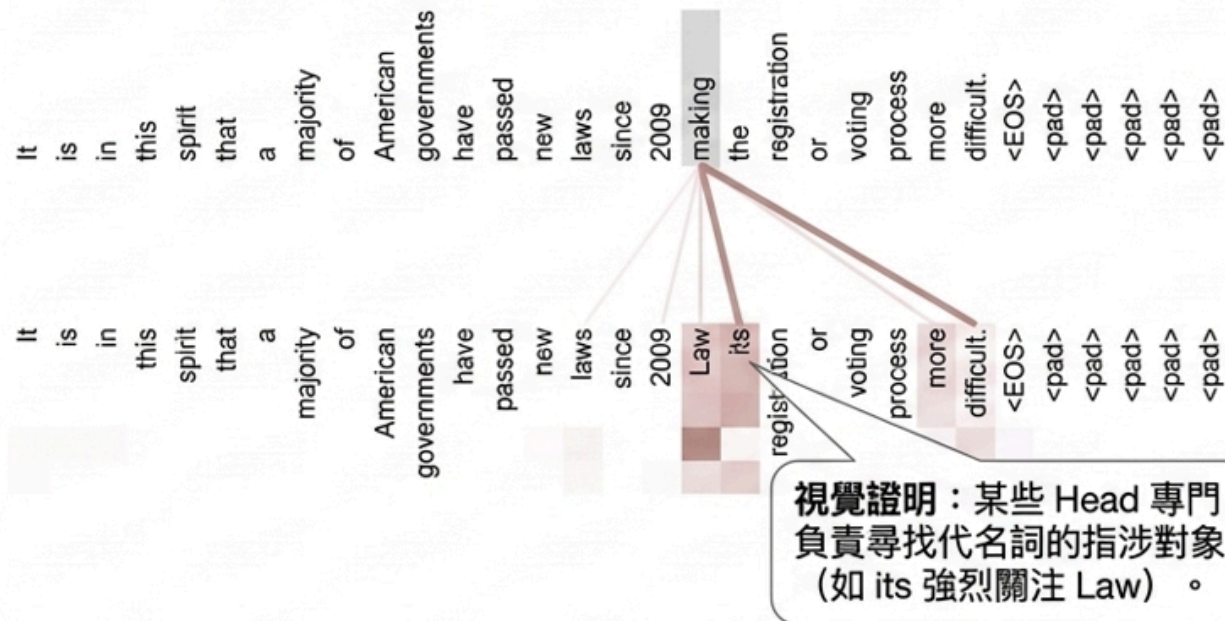
運作三部曲：

1. **配對 (MatMul)：** Q 與 K 計算內積 (Dot Product)，算出關聯分數。
2. **縮放與正規化 (Scale & Softmax)：** 除以 $\sqrt{d_k}$ 避免梯度消失，再通過 Softmax 轉為機率分佈的權重。
3. **提取 (MatMul with V)：** 依據算出的權重，將對應的 V 進行加權總和，產生最終輸出。

多頭注意力 (Multi-Head Attention)：捕捉多維度的特徵



- **單一頭的盲點**：如果只做一次 Attention，所有的關聯特徵會被平均化，失去細節。
- **分工合作 ($h=8$)**：將 Q 、 K 、 V 投影到 8 個不同的低維度空間平行執行。不同的 Head 會自動學會不同的任務，且運算成本與單頭幾乎相同。

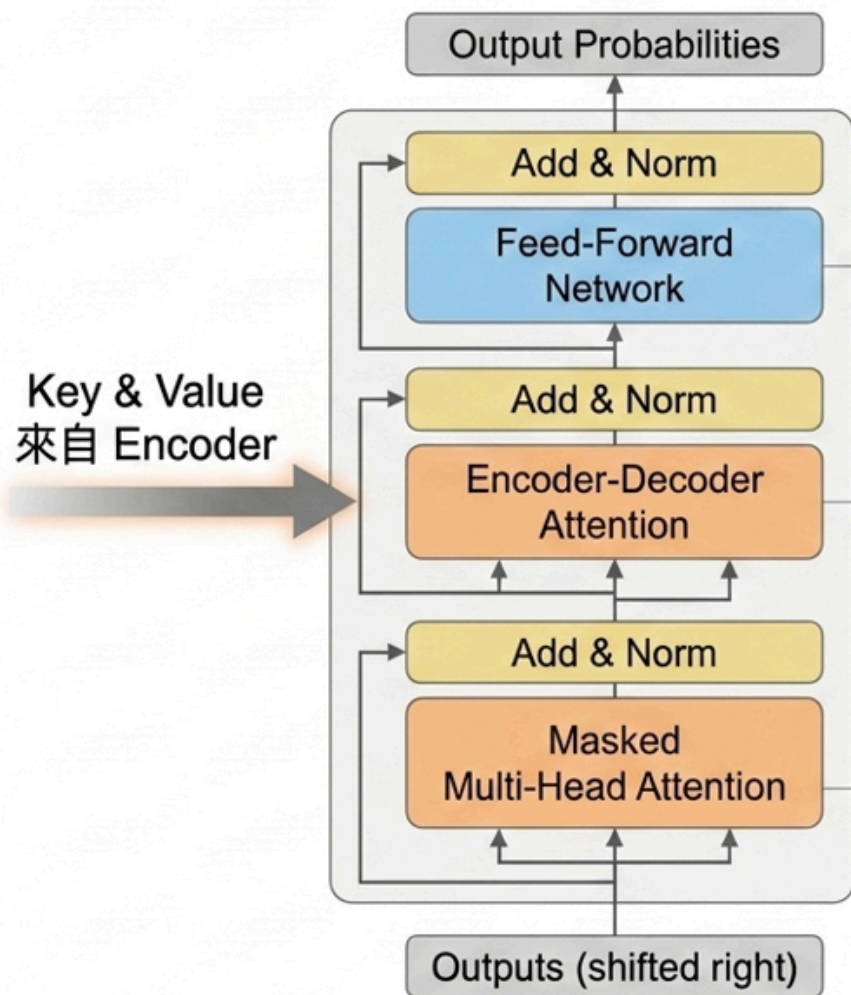


總結：Multi-Head Attention 讓模型同時看多種關係

- 一個 head 只能形成一套注意力分配，多個 head 可以平行學多種語意關係。

概念	單一 head	多個 head
關係類型	一套 attention pattern。	多套 attention pattern。
表示空間	所有資訊被迫放在同一個投影空間。	不同 head 可學不同投影空間。
輸出方式	直接得到 attention output。	Concat 各 head 結果，再經線性層混合。
課堂直覺	一位助教看句子。	多位助教分別看語法、語意、位置關係。

Masked and Cross Attention:



3. 特徵轉換 (FFN)

執行與 Encoder 完全相同的非線性特徵轉換。

2. 跨界關聯 (Cross Attention)

跨界尋找關聯。Query 來自上一層的 Decoder，但 Key 與 Value 來自 Encoder 的最終輸出。讓解碼器在生成每個字時，精準對焦輸入序列。

1. 遮蔽注意力 (Masked Attention)

自迴歸限制：生成第 i 個位置的輸出時，絕對不能偷看 $i + 1$ 以後的未來資訊。實作上是將非法連線的數值設為負無限大 $(-\infty)$ 。

四、Positional Encoding

- Attention 在運作時並沒有參考位置資訊，因此同樣字組的文字經 attention 運作的結果是相同的。為此 attention 需要額外位置訊息，才能知道 token 的先後順序。

同樣的詞集合

我

不

喜歡

貓

不同的語意

貓

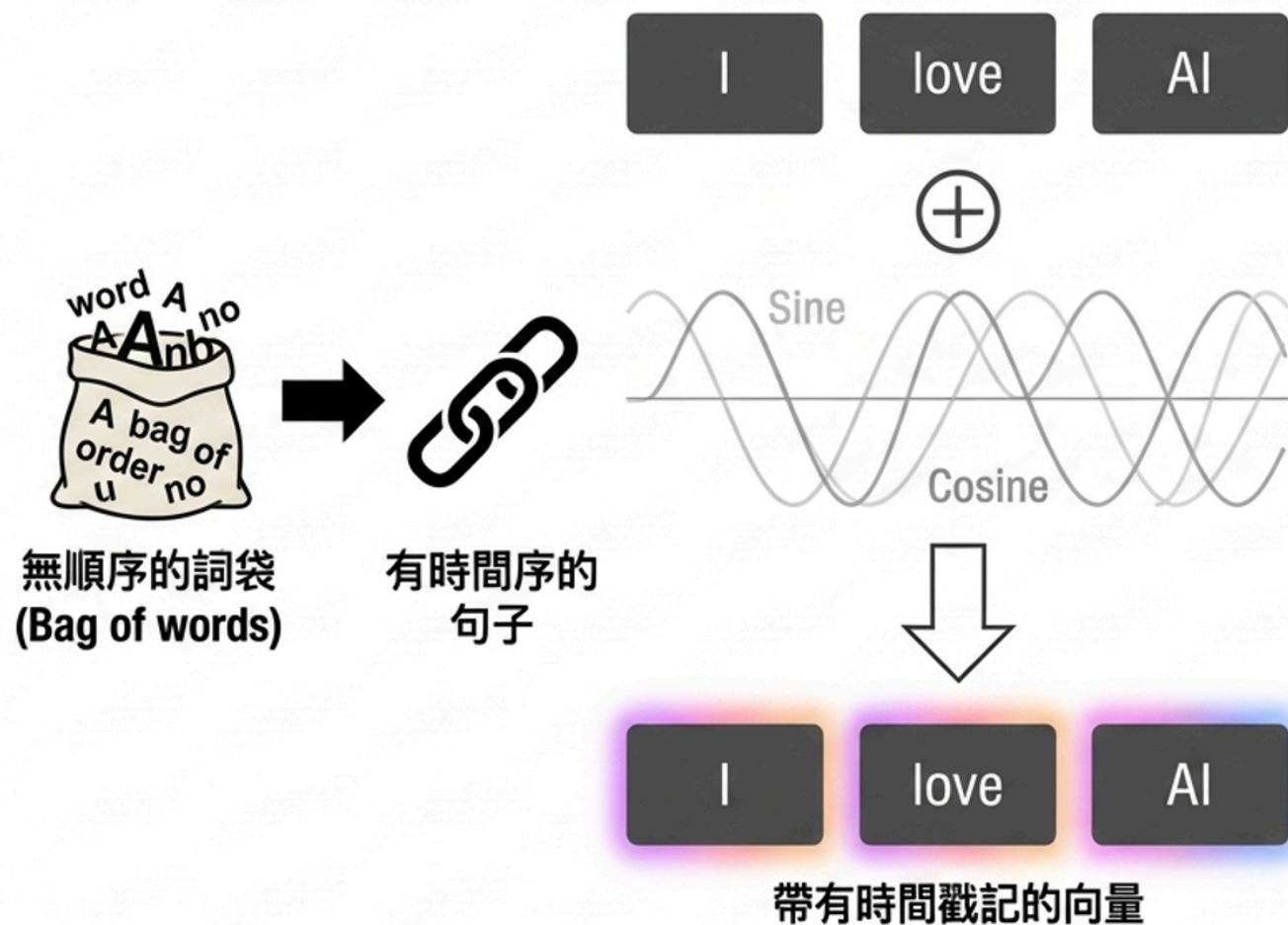
不

喜歡

我

- 如果模型只看 token 內容，這兩句話會很像。加入位置向量後，每個 token embedding 都帶著「我在第幾個位置」的訊息，模型才能分辨語序。

位置編碼的運作機制：



✂ 純注意力的缺陷：**Self-Attention** 本身是一個「詞袋模型」，打亂輸入詞彙的順序，輸出結果依然相同。它毫無序列順序的概念。

🧩 完美的幾何解決方案：

📌 • 不需要遞迴！直接將「位置資訊」作為一組加法訊號 (Additive signal) 注入輸入向量的底部。

$f(x)$ • 採用不同頻率的正弦 (sin) 與餘弦 (cos) 函數。

💡🧠 優勢：波長的幾何級數特性使得對應偏移量 k 的位置編碼可以表達成線性函數，使模型輕鬆學習到相對位置 (Relative positions) 的關聯，且能泛化至超長序列。

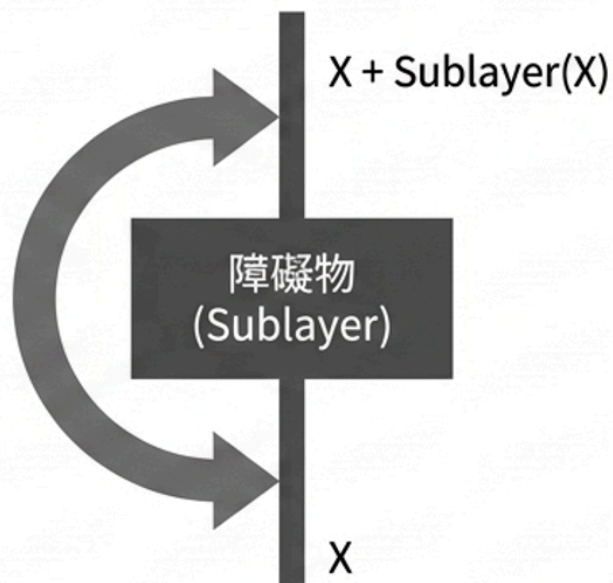
五、Residual 與 Layer Normalization

- 深層模型要穩定訓練，需要保留原訊息並控制表示尺度。



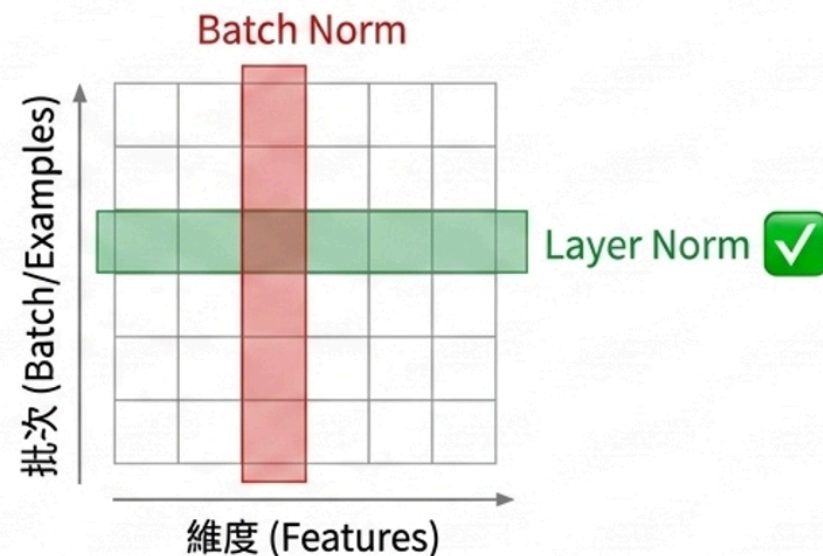
穩定訓練的護城河：Add & Norm

Residual Connection (殘差連接 - Add)



- 將輸入直接與子層的輸出相加。
- 作用：建立一條資訊直通的高速公路，解決深層神經網路的梯度消失問題，確保低層特徵不會在運算中遺失。

Layer Normalization (層正規化 - Norm)



- 不採用 Batch Normalization，而是針對「單一資料樣本」的所有特徵維度計算 Mean 與 Standard Deviation。
- 作用：序列長度變化極大時，不受 Batch 限制，確保特徵尺度統一，穩定模型的訓練與收斂速度。

Add & Norm 的常見結構：

- 每個子層外面包一個 residual connection 和 layer normalization

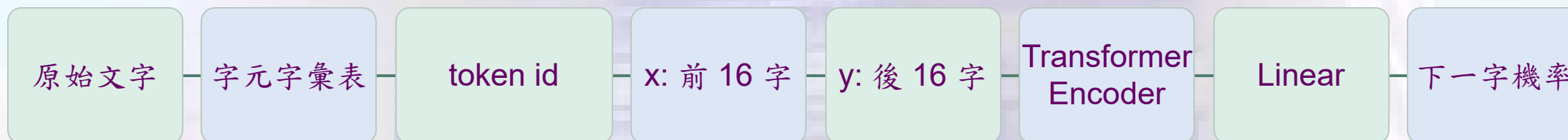
機制	直覺	訓練上的好處
Residual Connection	輸出 = 原本 x + 子層學到的修正。	保留訊息、改善梯度流動。
Layer Normalization	對每個 token 的 hidden dimensions 標準化。	穩定尺度、減少訓練震盪。
Dropout	訓練時隨機關掉部分 activations。	降低過度依賴與 overfitting。
Feed Forward	對每個位置獨立做 MLP。	增加非線性與轉換能力。

六、Transformer 的訓練

- 輸入與輸出：
 - 不同的應用其輸入與輸出都不相同
 - 文字接龍：輸入是一串文字，輸出是下一個字
 - 中翻英：輸入是中文，輸出是英文
- 分類問題：
 - 多數的 transformer 應用其輸出都是一個個的token
 - Token 是不連續的元素且個數是有限的，因此 transformer 處理的問題本質上可以看成是分類問題
- 損失函數：
 - 因此多數 transformer 問題都是採用 cross-entropy 作為損失函數，並用一般常見的優化器進行訓練

七、範例一：文字接龍

- 應用場域：給定一串文字，預測下一個中文字。
- 訓練目標：輸入一段長度 `block_size` 的文字，答案是右移一格的文字。



- 重點：這是一個只有 **encoder** 的範例，因此對於 **transformer** 的應用而言，不一定 **encoder** 及 **decoder** 都需要。

1. 匯入相關套件以及設定簡單的訓練資料

```
import torch
import torch.nn as nn
import math
from collections import Counter
```

```
text = """
```

機器學習是人工智慧的重要領域。

深度學習可以從大量資料中學習規律。

Transformer 是一種處理序列資料的模型。

它使用注意力機制來理解文字之間的關係。

語言模型可以根據前面的文字預測下一個字。

學生可以透過簡單範例理解 Transformer 的基本架構。

```
"""
```

2. 建立字彙表 (Tokenization)

```
counter = Counter(text) # 計算每個字的出現次數
chars = [ch for ch, count in counter.most_common()] # 依照出現次數排序
stoi = {ch: i for i, ch in enumerate(chars)} # 建立文字到數字的對應 (stoi 可把字轉成 id)
itos = {i: ch for ch, i in stoi.items()} # 建立數字到文字的對應 (itos 可把 id 轉回字)

vocab_size = len(chars)

data = torch.tensor([stoi[ch] for ch in text], dtype=torch.long) # 將文字轉成 token id

print("vocab size:", vocab_size)
print("data shape:", data.shape)
print(chars)
print(stoi['機']) # 顯示'機'這個字的編碼
print(itos[stoi['器']]) # 驗證 stoi 及 itos 兩個編碼表互為反函數
print(data) # 顯示整段文字的 token id 序列
```

3. 建立取得批次資料的函數 (get_batch)

- x 是目前看到的文字， y 是下一步要預測的答案

```
block_size = 16 # block_size 控制 context 長度  
batch_size = 8 # batch_size 控制一次訓練幾段文字
```

```
def get_batch():  
    ix = torch.randint(0, len(data) - block_size - 1, (batch_size,))  
    x = torch.stack([data[i:i+block_size] for i in ix])  
    y = torch.stack([data[i+1:i+block_size+1] for i in ix]) # y 比 x 右移一格  
    return x, y
```

```
x, y = get_batch()  
print(x.shape, y.shape)  
print("x[0] =", "".join(itos[i.item()] for i in x[0]))  
print("y[0] =", "".join(itos[i.item()] for i in y[0]))
```


4. 建立 PositionEncoding 類別

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model) # 建立一個形狀為 [max_len, d_model] 的 2D 張量

        # 生成一維序列 [0, 1, 2, ..., max_len-1]後再轉成 [max_len, 1] 的 2D 張量
        position = torch.arange(0, max_len).unsqueeze(1) # unsqueeze 可插入一個新維度

        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        ) # 計算 Transformer 論文中的分母縮放項
        pe[:, 0::2] = torch.sin(position * div_term) # 計算偶數通道的正弦值
        pe[:, 1::2] = torch.cos(position * div_term) # 計算奇數通道的餘弦值
        self.register_buffer("pe", pe.unsqueeze(0)) # 將 pe 插入一個 batch 維度
        # 將 pe 註冊為 nn.Module 的 Buffer，讓它不計算梯度，但可一起移到 cuda 又能支援存檔

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]
```

5. 建立 transformer 類別

```
class TinyTransformerLM(nn.Module):
    def __init__(self, vocab_size, d_model=64, nhead=4, num_layers=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model) # 建立「單字ID->高維向量」的離散查找表
        self.pos_encoder = PositionalEncoding(d_model) # 建立位置編碼類別
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, dim_feedforward=128, # Feed-Forward 的維度為 128
            dropout=0.1, batch_first=True # 第一個維度屬於 batch 的 size
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers) # num_layers 代表有幾個 block
        self.fc = nn.Linear(d_model, vocab_size)
    def forward(self, x):
        x = self.embedding(x)
        x = self.pos_encoder(x)
        x = self.transformer(x)
        return self.fc(x)
```

6. 設定損失函數及優化器後，開始進行訓練

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = TinyTransformerLM(vocab_size).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

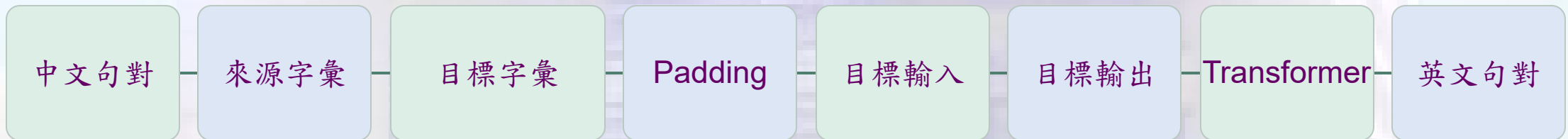
for step in range(1000):
    x, y = get_batch()
    x = x.to(device)
    y = y.to(device)
    logits = model(x)
    loss = loss_fn(logits.reshape(-1, vocab_size), y.reshape(-1)) # 將資料攤平後計算 loss
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 100 == 0:
        print(step, loss.item())
```

7. 定義生成函數並進行測試

```
def generate(model, start_text, max_new_tokens=80):  
    model.eval()  
    ids = [stoi[ch] for ch in start_text] # 將輸入文字轉成 token id 序列  
    x = torch.tensor(ids, dtype=torch.long).unsqueeze(0).to(device) # 插入 batch 維度  
    for _ in range(max_new_tokens):  
        x_cond = x[:, -block_size:] # 因長度有限制，故只保留最後 block_size 個字作為輸入  
        logits = model(x_cond)  
        next_logits = logits[:, -1, :] # 只抓出最後一個位置 (-1) 對字典中各個字詞的分數  
        probs = torch.softmax(next_logits, dim=-1) # 將得分透過 softmax 函數轉為機率  
        next_id = torch.multinomial(probs, num_samples=1) # 依照機率分佈抽樣(不是最大機率)  
        x = torch.cat([x, next_id], dim=1) # 將新抽樣出的單字ID拼接回原本序列的末端  
  
    result = "".join(itos[i] for i in x[0].tolist()) # 將所有字元串接成完整的字串  
    return result  
  
print(generate(model, "機器學習", 80)) # 開始進行文字接龍，"機器學習"為起始文字，最多80個字
```

八、範例二：中翻英

- 應用場域：給定一串中文字，翻成英文。
- 訓練目標：source 是中文，target 是英文。Decoder 訓練時使用 teacher forcing。



- 重點：和文字接龍不同，翻譯需要 encoder 讀中文，decoder 產生英文，因此 mask 與 padding 的角色更明顯。

1. 匯入相關套件以及中英文語句資料。

```
import torch
import torch.nn as nn
import math

pairs = [
    ("我愛你", "i love you"),
    ("我喜歡貓", "i like cats"),
    ("我喜歡狗", "i like dogs"),
    ("他是學生", "he is a student"),
    ("她是老師", "she is a teacher"),
    ("今天天氣好", "the weather is good today"),
    ("我正在學習", "i am learning"),
    ("我學習機器學習", "i study machine learning"),
    ("這是一本書", "this is a book"),
    ("你喜歡音樂", "you like music"),
]
```

2. 建立來源語言(中文)特殊 token 及字彙表

```
SRC_PAD = "<pad>" # 中文端 padding token，用來把不同長度的中文句子補成一樣長。  
SRC_UNK = "<unk>" # 中文端 UNK token，是 unknown 的意思，用來表示「字彙表裡沒有的字」。  
  
src_chars = sorted(list(set("".join(src for src, tgt in pairs)))) # 建立中文字彙表  
  
src_itos = [SRC_PAD, SRC_UNK] + src_chars # 在中文字彙表中加入特殊 token  
src_stoi = {ch: i for i, ch in enumerate(src_itos)} # 建立中文反向字彙表  
  
src_pad_id = src_stoi[SRC_PAD] # 取得 padding token id  
src_unk_id = src_stoi[SRC_UNK] # 取得 UNK token id  
  
print(src_stoi)
```

3. 建立目標語言(英文)特殊 token 及字彙表

```
TGT_PAD = "<pad>" # 英文端 padding token，用來把不同長度的中文句子補成一樣長。
TGT_UNK = "<unk>" # 英文端 UNK token，是 unknown 的意思，用來表示「字彙表裡沒有的字」。
TGT_SOS = "<sos>" # Start token，讓 decoder 知道起點
TGT_EOS = "<eos>" # End token，讓 decoder 知道應該停止了

tgt_chars = sorted(list(set("".join(tgt for src, tgt in pairs)))) # 建立英文字彙表

tgt_itos = [TGT_PAD, TGT_UNK, TGT_SOS, TGT_EOS] + tgt_chars # 在英文字彙表中加入特殊 token
tgt_stoi = {ch: i for i, ch in enumerate(tgt_itos)} # 建立英文反向字彙表

tgt_pad_id = tgt_stoi[TGT_PAD] # 取得 padding token id
tgt_unk_id = tgt_stoi[TGT_UNK] # 取得 UNK token id
tgt_sos_id = tgt_stoi[TGT_SOS] # 取得 start token id
tgt_eos_id = tgt_stoi[TGT_EOS] # 取得 end token id

print(tgt_stoi)
```

4. 建立來源及目標語言編碼函數以及文句補 padding token 函數

```
def encode_src(text): # 將中文文句轉成 token id 序列
    return [src_stoi.get(ch, src_unk_id) for ch in text]
def encode_tgt(text): # 將英文句子轉成 token id 序列
    return [tgt_sos_id] + [tgt_stoi.get(ch, tgt_unk_id) for ch in text] + [tgt_eos_id]
print(encode_src("我愛你"))
print(encode_tgt("i love you"))

def pad_sequences(sequences, pad_id): # 把句子加上 padding token 變成固定長度
    max_len = max(len(seq) for seq in sequences)
    result = []

    for seq in sequences:
        padded = seq + [pad_id] * (max_len - len(seq))
        result.append(padded)

    return torch.tensor(result, dtype=torch.long)
```

5. 建立來源及目標語言的訓練資料

```
src_sequences = [encode_src(src) for src, tgt in pairs] # 將中文文句轉成 token id 序列
tgt_sequences = [encode_tgt(tgt) for src, tgt in pairs] # 將英文句子轉成 token id 序列
print(src_sequences)
print(tgt_sequences)
src_batch = pad_sequences(src_sequences, src_pad_id) # 將所有中文文句變成相同長度
tgt_batch = pad_sequences(tgt_sequences, tgt_pad_id) # 將所有英文句子變成相同長度
print(src_batch.shape)
print(tgt_batch.shape)

# 建立訓練時 decoder 所需的輸入及輸出 token id 序列資料 (使用 teacher forcing 機制)
tgt_input = tgt_batch[:, :-1] # 選取從第 0 個位置到倒數第 2 個位置 (即去掉最後一個 token)
tgt_output = tgt_batch[:, 1:] # 選取從第 1 個位置到最後一個位置 (即去掉第一個 token)
print(tgt_input.shape)
print(tgt_output.shape)
```


6. 建立 PositionEncoding 類別

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=500):
        super().__init__()
        pe = torch.zeros(max_len, d_model) # 建立一個形狀為 [max_len, d_model] 的 2D 張量

        # 生成一維序列 [0, 1, 2, ..., max_len-1]後再轉成 [max_len, 1] 的 2D 張量
        position = torch.arange(0, max_len).unsqueeze(1) # unsqueeze 可插入一個新維度

        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        ) # 計算 Transformer 論文中的分母縮放項
        pe[:, 0::2] = torch.sin(position * div_term) # 計算偶數通道的正弦值
        pe[:, 1::2] = torch.cos(position * div_term) # 計算奇數通道的餘弦值
        self.register_buffer("pe", pe.unsqueeze(0)) # 將 pe 插入一個 batch 維度
        # 將 pe 註冊為 nn.Module 的 Buffer，讓它不計算梯度，但可一起移到 cuda 又能支援存檔

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]
```

7. 建立 transformer 類別

```
class TransformerTranslator(nn.Module):
    def __init__(
        self, src_vocab_size, tgt_vocab_size, d_model=64, nhead=4,
        num_encoder_layers=2, num_decoder_layers=2, dim_feedforward=128, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.src_embedding = nn.Embedding(src_vocab_size, d_model) # 建立來源語言的離散查找表
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model) # 建立目標語言的離散查找表
        self.pos_encoder = PositionalEncoding(d_model) # 建立 encoder 位置編碼類別
        self.pos_decoder = PositionalEncoding(d_model) # 建立 decoder 位置編碼類別
        self.transformer = nn.Transformer( # 設定 transformer 相關參數 (幾個head, 幾個block)
            d_model=d_model, nhead=nhead, num_encoder_layers=num_encoder_layers,
            num_decoder_layers=num_decoder_layers, dim_feedforward=dim_feedforward,
            dropout=dropout, batch_first=True
        )
        self.fc_out = nn.Linear(d_model, tgt_vocab_size) # 最後的線性轉換層
```

```
def forward(self, src, tgt, src_key_padding_mask=None,
            tgt_key_padding_mask=None, memory_key_padding_mask=None, tgt_mask=None ):

    src_emb = self.src_embedding(src) * math.sqrt(self.d_model)
    tgt_emb = self.tgt_embedding(tgt) * math.sqrt(self.d_model)
    src_emb = self.pos_encoder(src_emb) # 加入 encoder 位置編碼
    tgt_emb = self.pos_decoder(tgt_emb) # 加入 decoder 位置編碼

    output = self.transformer(
        src_emb,
        tgt_emb,
        tgt_mask=tgt_mask, # 避免 decoder 偷看未來 token
        src_key_padding_mask=src_key_padding_mask, # 遮蔽來源語言的 padding token
        tgt_key_padding_mask=tgt_key_padding_mask, # 遮蔽目標語言的 padding token
        memory_key_padding_mask=memory_key_padding_mask # 在解碼時，遮蔽來源語言的padding token
    )

    logits = self.fc_out(output)
    return logits
```

8. 定義 tgt_mask 以及 padding_mask 的建立函數

```
def generate_square_subsequent_mask(size):  
    # 建立一個方陣，其中主對角線(diagonal=1 表示不包含主對角線)以上的數值為 1，其餘為 0  
    mask = torch.triu(torch.ones(size, size), diagonal=1)  
    # 將 mask 矩陣中數值為 1 的位置填為 -inf (負無窮大)，表示這個位置對應的 token 會被遮蔽  
    mask = mask.masked_fill(mask == 1, float("-inf"))  
    return mask  
  
# 若輸入 token id 串列中的 id == pad_id，將此位置設為 true，其餘為 false  
def create_padding_mask(seq, pad_id):  
    return seq == pad_id
```

9. 建立 transformer 物件實體並設定優化器以及損失函數

```
device = "cuda" if torch.cuda.is_available() else "cpu"

model = TransformerTranslator( # 建立 transformer 物件實體
    src_vocab_size=len(src_itos), # Encoder輸入序列的長度
    tgt_vocab_size=len(tgt_itos), # Decoder輸入序列的長度
    d_model=64, # token 向量維度
    nhead=4, # 多頭注意力中的 head 個數
    num_encoder_layers=2, # Encoder block 個數
    num_decoder_layers=2 # Decoder block 個數
).to(device)

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3) # 採用 AdamW 優化器

loss_fn = nn.CrossEntropyLoss(ignore_index=tgt_pad_id) # 使用 cross entropy 損失函數
```

10. 建立訓練迴圈並開始進行訓練

```
for epoch in range(1000):
    model.train()
    src_batch_dev = src_batch.to(device)
    tgt_input_dev = tgt_input.to(device)
    tgt_output_dev = tgt_output.to(device)
    tgt_mask = generate_square_subsequent_mask(tgt_input.size(1)).to(device)
    src_padding_mask = create_padding_mask(src_batch, src_pad_id).to(device)
    tgt_padding_mask = create_padding_mask(tgt_input, tgt_pad_id).to(device)
    logits = model(src_batch_dev, tgt_input_dev,
                    src_key_padding_mask=src_padding_mask, tgt_key_padding_mask=tgt_padding_mask,
                    memory_key_padding_mask=src_padding_mask, tgt_mask=tgt_mask)
    loss = loss_fn(logits.reshape(-1, len(tgt_itos)), tgt_output_dev.reshape(-1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 100 == 0:
        print(f"epoch {epoch}, loss = {loss.item():.4f}")
```


11. 利用已訓練好的 transformer 模型，建立中翻英的翻譯函數

```
def translate(model, src_text, max_len=50):
    model.eval()
    src_ids = encode_src(src_text) # 將輸入文字轉成 token id 串列
    src = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device) # 加一維度並轉成張量
    src_padding_mask = create_padding_mask(src, src_pad_id).to(device)
    tgt_ids = [tgt_sos_id] # Decoder 的輸入一開始只有起始 token
    for _ in range(max_len): # 逐一處理每一個字元
        tgt = torch.tensor(tgt_ids, dtype=torch.long).unsqueeze(0).to(device) # 加一維度並轉成張量
        tgt_mask = generate_square_subsequent_mask(tgt.size(1)).to(device)
        logits = model(src, tgt, src_key_padding_mask=src_padding_mask,
                        tgt_key_padding_mask=None, memory_key_padding_mask=src_padding_mask, tgt_mask=tgt_mask)
        next_logits = logits[:, -1, :] # 取出 decoder 最後一個輸出對應的 token 分數
        next_id = torch.argmax(next_logits, dim=-1).item() # 找出 token 對應的 id
        if next_id == tgt_eos_id: # 若結果等於結束 token，則停止運算
            break
        tgt_ids.append(next_id) # 將得到的結果加入目前的 tgt_ids 串列
    result = "".join(tgt_itos[i] for i in tgt_ids[1:]) # 最後將 tgt_ids 串列轉成對應的文字
    return result
```

12. 進行中翻英模型的測試

```
print(translate(model, "我愛你"))  
print(translate(model, "我喜歡貓"))  
print(translate(model, "他是學生"))  
print(translate(model, "這是一本書"))  
print(translate(model, "我喜歡機器學習"))
```

本章總結

- 學習重點檢查表

問題	回答
Transformer 適合哪些任務？	輸入與輸出都是序列的任務，例如翻譯、語音辨識、摘要、聊天。
Self-Attention 在做什麼？	每個 token 用 Q、K、V 計算與其他 token 的關係，再加權收集資訊。
為什麼要 Positional Encoding？	Attention 本身不知道順序，需要額外位置訊號。
Decoder 為什麼要 mask？	生成第 t 個 token 時不能偷看未來答案。